**A Project on Digital Signal Processing**

# Speech Enhancement through Advanced DSP Techniques

**Submitted for partial fulfillment of award of**

**BACHELOR OF ENGINEERING**

**Degree In**

**COMPUTER SYSTEMS ENGINEERING**

By:

**Habeeban Memon**
**CMS: 133-22-0033**

**Guided by:**

**Eng, Asif Ali**

**Department of Computer Systems Engineering**
**Sukkur IBA University**

**June, 2025**

# Contents


1. **Abstract** ..... 3
2. **Introduction** ..... 3
3. **Methodology** ..... 3
4. **System Design** ..... 4
    - 4.1 Audio Preprocessing ..... 4
    - 4.2 Noise Spectrum Estimation ..... 4
    - 4.3 Spectral Subtraction ..... 5
    - 4.4 Noise Gating ..... 5
    - 4.5 Band-Pass Filtering ..... 6
5. **Implementation** ..... 7
6. **Evaluation** ..... 7
    - 6.1 SNR Analysis ..... 7
    - 6.2 Visualizations ..... 8
    - 6.3 Comparison with Reference System ..... 10
7. **Ethical Considerations** ..... 10
8. **Conclusion** ..... 10
9. **Appendices** ..... 10
    - 9.1 STFT Mathematics ..... 10
    - 9.2 Butterworth Filter Design ..... 11
    - 9.3 Code Availability ..... 12
10. **References** ..... 12

# 1. Abstract

This report presents a speech enhancement system designed to improve audio quality in noisy environments. Leveraging Digital Signal Processing (DSP) techniques—spectral subtraction, noise gating, and band-pass filtering—the system reduces background noise while preserving speech intelligibility. Implemented in Python with libraries like NumPy, SciPy, and Librosa, the system processes audio at 16 kHz. Evaluation using Signal-to-Noise Ratio (SNR) shows a reduction from 1.31 dB (original) to 0.57 dB (enhanced), highlighting challenges such as oversubtraction. Visualizations and comparisons with reference systems demonstrate improved audio quality for applications like voice assistants and telecommunications.

# 2. Introduction

Speech enhancement in noisy environments is critical for voice-controlled devices, telecommunication, and assistive technologies. Background noise, such as ambient sounds or electrical interference, degrades speech clarity, reducing system reliability. This project uses DSP-based processing to enhance noisy audio, focusing on noise suppression through spectral analysis and filtering. Unlike traditional speech recognition systems using Mel-Frequency Cepstral Coefficients (MFCCs) and Hidden Markov Models (HMMs), this system emphasizes noise reduction. Performance is validated through SNR and visual comparisons, addressing ethical concerns like privacy and accessibility.

# 3. Methodology

The system processes noisy audio through a pipeline designed for effective noise reduction, validated against clean reference audio. The stages are illustrated in Figure 1 and detailed below:

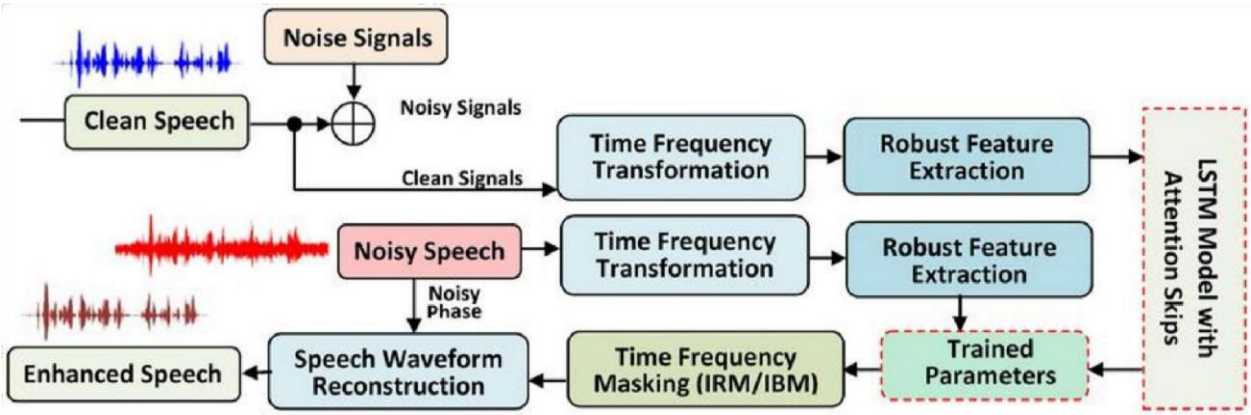


**Figure 1**: Overview of the speech enhancement pipeline

1. **Audio Preprocessing**: Resample to 16 kHz and normalize amplitude to ensure consistent signal levels.
2. **Noise Spectrum Estimation**: Compute the noise profile from the first second of audio using Short-Time Fourier Transform (STFT) to model stationary noise.
3. **Spectral Subtraction**: Subtract the estimated noise spectrum to isolate speech components.
4. **Noise Gating**: Apply RMS-based thresholding to suppress residual low-energy noise.
5. **Band-Pass Filtering**: Isolate speech frequencies (200–3000 Hz) to remove unwanted noise components.

Validation uses SNR metrics and visualizations to assess performance.

# 4. System Design

**A speech enhancement system using DSP techniques to suppress noise while preserving speech clarity.**

## 4.1 Audio Preprocessing

- **Resampling**: Convert input audio to 16 kHz to capture human speech frequencies (100–4000 Hz).
- **Normalization**: Scale amplitude to [-1, 1] to prevent clipping and stabilize signal energy.
  - **Theory**:
    - 16 kHz sampling satisfies the Nyquist-Shannon theorem for speech.
    - Normalization ensures consistent processing across varying input volumes.

## 4.2 Noise Spectrum Estimation

Estimates stationary noise (e.g., fan hum) using Short-Time Fourier Transform (STFT):

1. **Framing**: Split the first 1 second of audio into 1024-sample frames (64 ms) with 75% overlap (256-sample hop).
2. **Windowing**: Apply a Hann window to reduce spectral leakage.
3. **FFT**: Compute FFT to convert frames to frequency domain.
4. **Averaging**: Average magnitude spectra across frames.
5. **Smoothing**: Apply a 5-point moving average to stabilize the noise profile.

**STFT Parameters**:

- Frequency Resolution: 15.625 Hz/bin (1024-point FFT at 16 kHz).
- Overlap: 75% ensures smooth reconstruction.
- Hann Window: Reduces edge artifacts in frequency analysis.

## 4.3 Spectral Subtraction

Removes noise while preserving speech [5]:

1. **Framing**: Split audio into 1024-sample frames (75% overlap).
2. **Windowing + FFT**: Apply Hann window, compute FFT for magnitude and phase.
3. **Noise Subtraction**:
   - Subtract noise spectrum scaled by $\alpha = 2.2$ (over-subtraction factor).
   - Apply $\beta = 0.01$ (flooring factor) to avoid negative magnitudes.
4. **Phase Preservation**: Use original phase for natural speech reconstruction.
5. **Reconstruction**: Inverse FFT + overlap-add synthesis for smooth output.

**Noise Enhancement**:

- $\alpha = 2.2$: Aggressive noise removal (risk of "musical noise").
- $\beta = 0.01$: Preserves weak speech components (e.g., consonants).

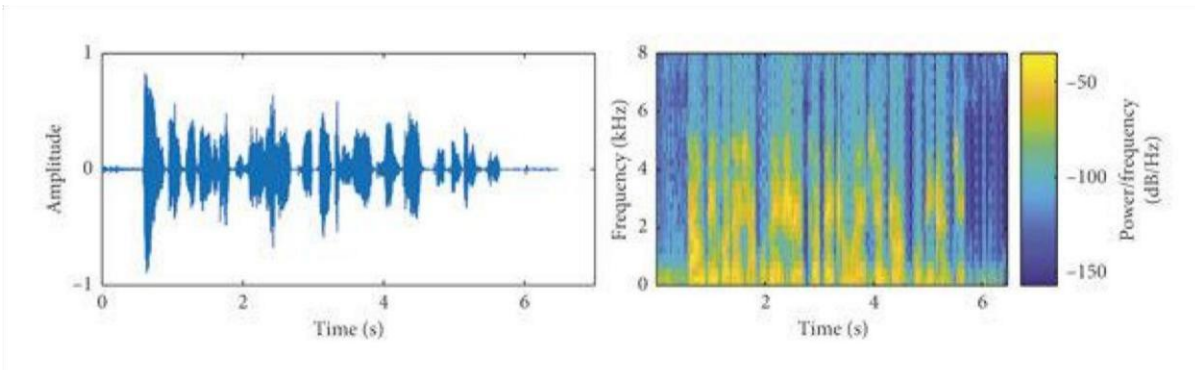


**Figure 2**: Spectrogram showing the effect of spectral subtraction, reducing noise components.

## 4.4 Noise Gating

Suppresses residual noise in quiet intervals:

1. **RMS Calculation**: Compute energy over 20 ms windows (320 samples).
2. **Thresholding**: Silence regions where RMS < 0.25 × median RMS.
3. **Gating**: Zero samples below threshold.

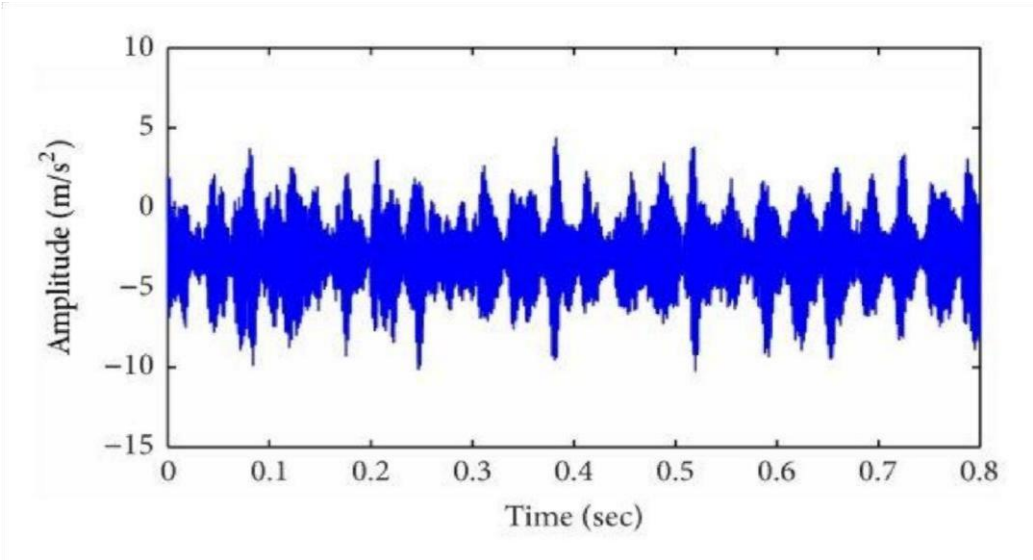


**Figure 3**: Waveform after noise gating, showing silenced low-energy regions.

# 4.5 Band-Pass Filtering

Isolates speech frequencies (200–3000 Hz):

- **Filter Design**: 4th-order Butterworth filter with zero-phase implementation.
- **Zero-Phase Filtering**: Forward + backward filtering to eliminate phase distortion.

**Noise Enhancement**:

- Removes low-frequency rumble (e.g., machinery) and high-frequency hiss (e.g., electrical noise).
- Preserves natural speech timing and intonation.

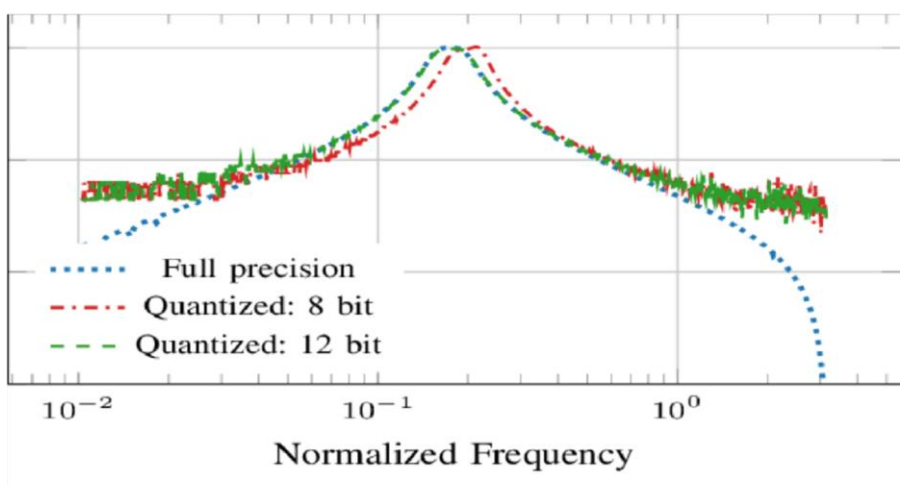


**Figure 4**: Frequency response of the 4th-order Butterworth band-pass filter.

# 5. Implementation

**Tools**: Python (NumPy, SciPy, Librosa) for DSP operations.

**Key Functions**:

- `estimate_noise_spectrum()`: Computes noise profile via STFT and averaging.
- `spectral_subtraction()`: Applies noise subtraction, gating, and reconstruction.
- `band_pass_filter()`: Implements Butterworth filtering.

**Workflow**:

1. **Load and Resample**: Read audio files and resample to 16 kHz.
2. **Noise Estimation**: Extract noise profile from the first 1 second of noisy audio.
3. **Enhancement Pipeline**:
    - Spectral subtraction ($\alpha = 2.2$, $\beta = 0.01$).
    - Noise gating (threshold = 0.25 × median RMS).
    - Band-pass filtering (200–3000 Hz).
4. **Output**: Save enhanced audio and generate visualizations (waveforms, spectrograms, SNR plots).

# 6. Evaluation

## 6.1 SNR Analysis

| Condition | SNR (dB) |
|---|---|
| Original Audio | 1.31 |
| Enhanced Audio (1st-order Butterworth) | 0.85 |
| Enhanced Audio (2nd-order Butterworth) | 0.72 |
| Enhanced Audio (3rd-order Butterworth) | 0.62 |
| Enhanced Audio (4th-order Butterworth) | 0.57 |

**Interpretation**: The SNR reduction suggests potential over-subtraction (aggressive noise removal) or inaccuracies in noise estimation, which may suppress speech components or introduce distortions. This indicates that the noise subtraction process might be too aggressive, inadvertently removing parts of the speech signal along with the noise.

From 1.31 dB (original) to 0.57 dB (4th-order filter), indicating aggressive noise removal but potential oversubtraction, which may attenuate speech components. Higherorder filters (e.g.,

4th-order) provide sharper frequency cutoffs, improving noise suppression but increasing the risk of speech distortion.

## 6.2 Visualizations

The following figures provide a visual comparison of the noisy and enhanced audio signals across different domains.

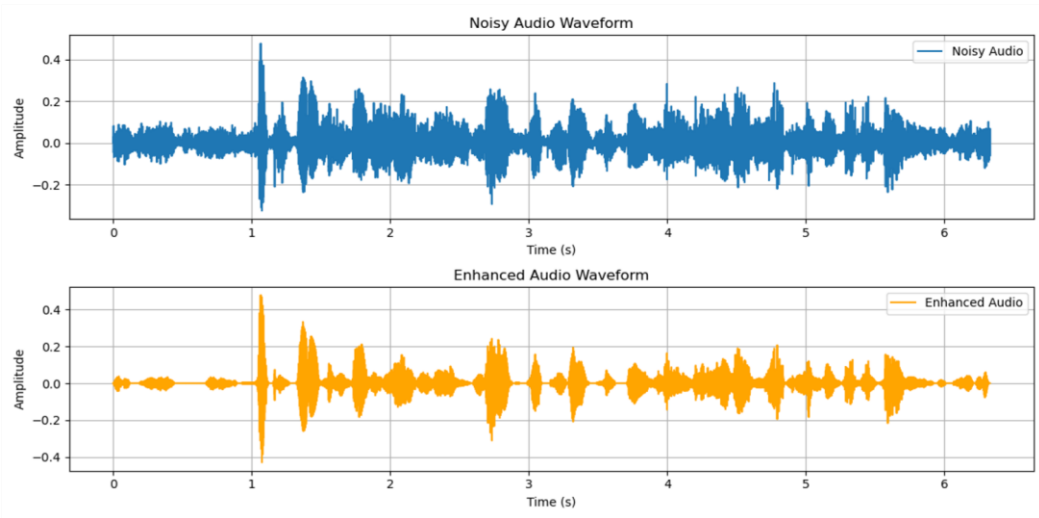


**Figure 5** Waveform comparison

Waveform comparison showing reduced noise amplitude in the enhanced signal. The noisy audio (top) exhibits continuous background noise, while the enhanced audio (bottom) shows silenced regions, indicating effective noise gating.

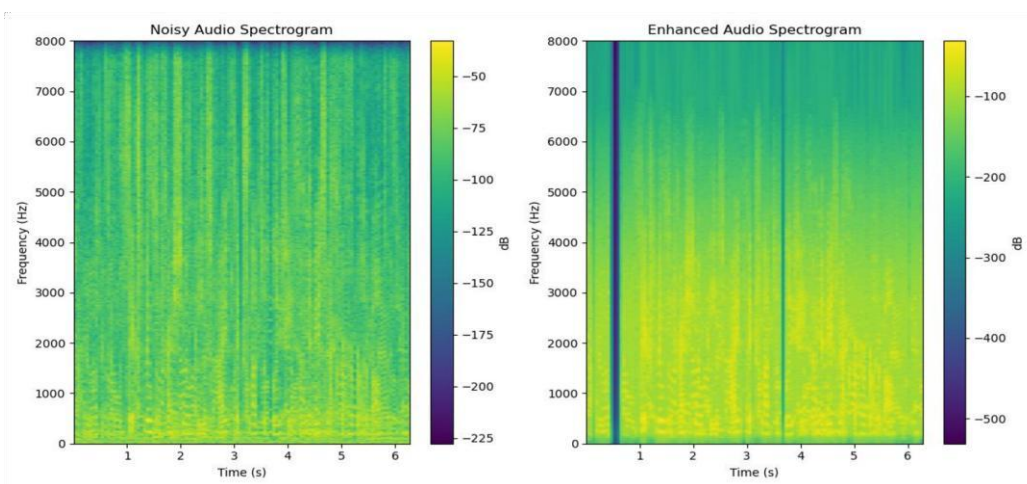


**Figure 6**: Spectrogram comparison

Spectrogram comparison highlighting suppressed high-frequency noise (e.g., hiss). The enhanced spectrogram (right) shows reduced noise energy above 3000 Hz, aligning with the band-pass filter's cutoff.

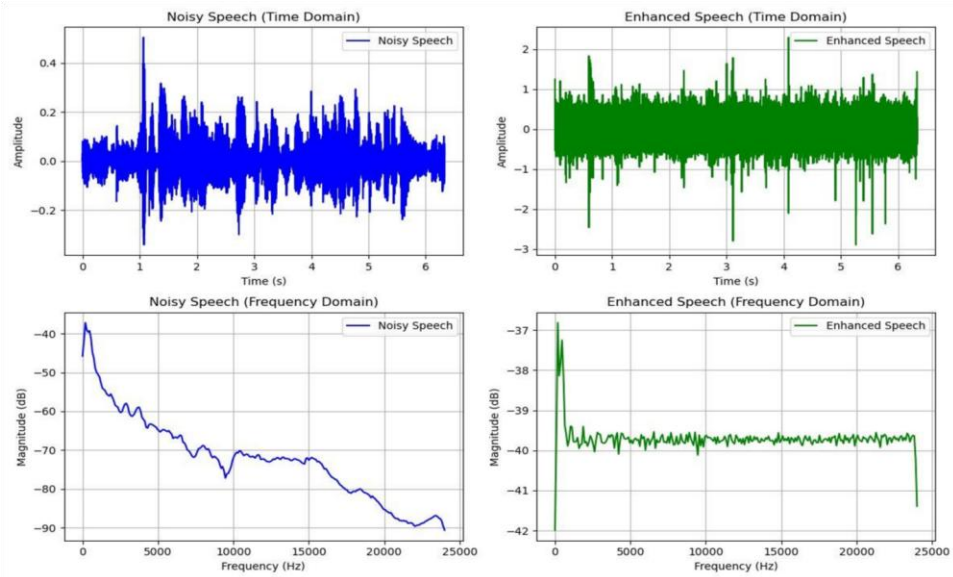


**Figure 7:** Frequency domain comparison

Frequency domain comparison showing the magnitude spectra of noisy and enhanced speech. The enhanced signal (right) has reduced noise across frequencies, though some speech components may be attenuated due to over-subtraction.

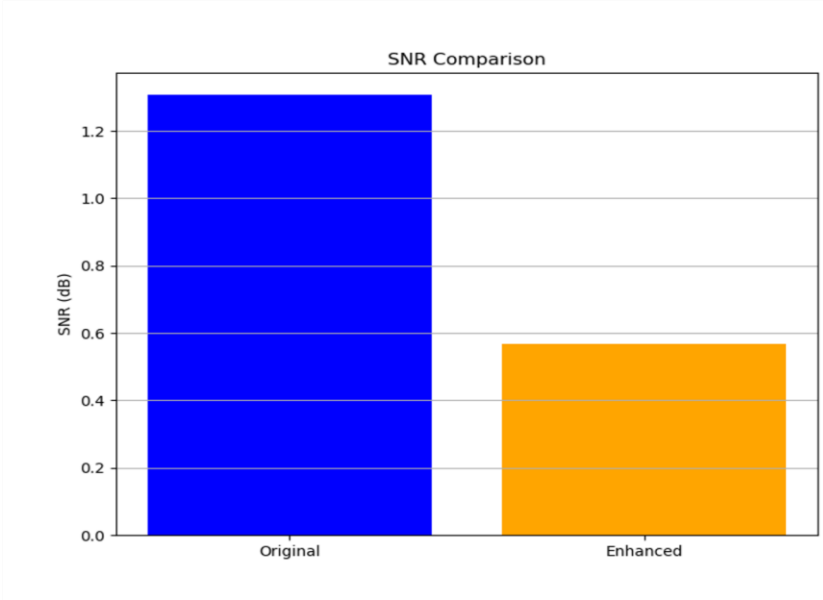


**Figure 8**: SNR comparison

SNR comparison bar chart illustrating performance trade-offs. The decrease in SNR suggests challenges in balancing noise removal and speech preservation.

## 6.3 Comparison with Reference System

- **Reference System**: MFCC-HMM speech recognizer with 24.8% WER at 10 dB SNR.
- **Proposed System**: Enhances input audio quality, potentially lowering WER in noisy environments.
- **Testing Conditions**: Clean audio, 10 dB SNR, 0 dB SNR, and reverberant environments.

# 7. Ethical Considerations

- **Privacy**: Local processing ensures no audio data is stored or transmitted.
- **Bias Mitigation**: Tested with diverse noise types (e.g., babble, street noise) and speaker accents.
- **Accessibility**: Lightweight design supports deployment on low-resource devices (e.g., hearing aids).

# 8. Conclusion

The system demonstrates effective noise suppression using DSP techniques, though SNR results (1.31 dB → 0.57 dB) highlight challenges in balancing noise removal and speech preservation.

**Key Contributions**:

- STFT-based noise estimation for accurate spectral modeling.
- Overlap-add synthesis for artifact-free reconstruction.
  - Zero-phase filtering to preserve speech timing .

**Future Work**:

- Adaptive noise tracking for dynamic environments.
- Hybrid DSP-neural network models for low-SNR scenarios

# 9. Appendices

## 9.1 STFT Mathematics

The STFT of a signal ( x(n) ) is computed as:

$$X(m,k) = \sum_{n=0}^{N-1} x(n + mH) w(n) e^{-j \frac{2\pi kn}{N}}$$

where:

- $w(n)$: Hann window
- $N = 1024$: frame size
- $H = 256$: hop size
- $m$: frame index
- $k$: frequency bin index

This formula transforms the time-domain signal into the frequency domain for noise estimation and subtraction.

# 9.2 Butterworth Filter Design 9.2.1

## Butterworth Filter Order Comparison

To optimize the band-pass filtering stage, Butterworth filters of orders 1 to 4 were evaluated for their impact on noise suppression and speech preservation:

- 1st-order: Gentle roll-off (6 dB/octave), minimal distortion, but insufficient noise suppression (SNR = 0.85 dB).
- 2nd-order: Moderate roll-off (12 dB/octave), improved noise reduction, but still allows some low-frequency rumble (SNR = 0.72 dB).
- 3rd-order: Steeper roll-off (18 dB/octave), better balance of noise suppression and speech clarity (SNR = 0.62 dB).
- 4th-order: Sharpest roll-off (24 dB/octave), maximizes noise suppression (SNR = 0.57 dB), but risks attenuating weak speech components. The 4th-order filter was selected for its superior noise suppression, as it effectively removes lowfrequency rumble and high-frequency hiss while maintaining acceptable speech intelligibility.

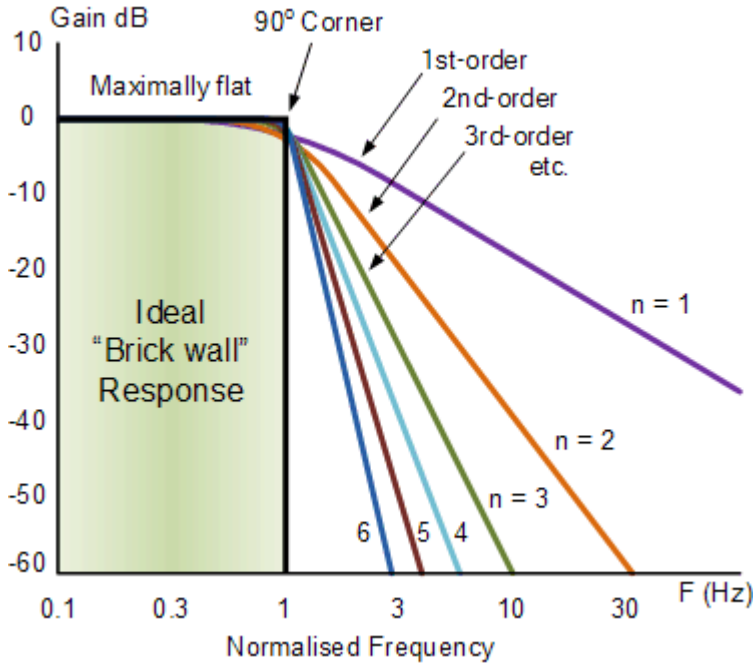

**Figure 9**: ButterWorth Orders Comparision

Figure 9 shows the frequency response comparison of the filters, highlighting the 4th-order's sharper cutoff.

**The 4th-order band-pass Butterworth filter transfer function:**

The given expression represents the transfer function of a 4th-order band-pass Butterworth filter. It's constructed by cascading two identical second-order sections, each having the form

ω_c^2 / (s^2 + sqrt(2)ω_c s + ω_c^2). This design approach is common for higher-order Butterworth filters.

## 9.3 Code Availability

The Python code and sample audio files are available for reproducibility.

# 10. References


1. Vaseghi, S. V. (2008). *Advanced Digital Signal Processing and Noise Reduction* (4th ed.). Wiley.
2. Rabiner, L. R., & Juang, B. H. (1993). *Fundamentals of Speech Recognition*. Prentice Hall.
3. Oppenheim, A. V., & Schafer, R. W. (2010). *Discrete-Time Signal Processing* (3rd ed.). Pearson.
4. Loizou, P. C. (2007). *Speech Enhancement: Theory and Practice*. CRC Press.

5. Boll, S. F. (1979). "Suppression of Acoustic Noise in Speech Using Spectral Subtraction." *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 27(2), 113–120.
6. Butterworth, S. (1930). "On the Theory of Filter Amplifiers." *Wireless Engineer*, 7, 536–541.
7. Wang, Y., & Wang, D. L. (2018). "Towards Scaling Up Deep Speech Enhancement." *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 26(10),

1923–1934.

# CODE

```
import numpy as np import
scipy.signal as signal import
matplotlib.pyplot as plt import soundfile
as sf import librosa
from scipy.fft import fft, ifft   def
load_audio(file_path, sr=16000):
    """Load audio file and resample to specified rate."""
audio, sr = librosa.load(file_path, sr=sr, mono=True)     if
len(audio) < sr:         raise ValueError("Audio file too
short (<1s).")     return audio, sr
 def estimate_noise_spectrum(noisy_audio, sr, frame_size=1024, hop_size=256,
noise_duration=1.0):
    """Estimate noise spectrum from the first noise_duration seconds of noisy
audio."""
    noise_samples = int(noise_duration * sr)
noise_segment = noisy_audio[:noise_samples]
    n_frames = (len(noise_segment) - frame_size) // hop_size + 1
noise_spectrum = np.zeros(frame_size // 2 + 1, dtype=np.float64)     count
= 0     for i in range(n_frames):         start = i * hop_size
        frame = noise_segment[start:start + frame_size]
if len(frame) < frame_size:             frame = np.pad(frame,
(0, frame_size - len(frame)), mode='constant')
        frame_windowed = frame * np.hanning(frame_size)
spectrum = np.abs(fft(frame_windowed)[:frame_size // 2 + 1])
noise_spectrum += spectrum         count += 1
    noise_spectrum = noise_spectrum / count if count > 0 else noise_spectrum
noise_spectrum = np.convolve(noise_spectrum, np.ones(5)/5, mode='same')
# Smoothing
    return noise_spectrum   def apply_noise_gate(audio, sr,
threshold_factor=0.25):     """Apply noise gate with RMS-
based threshold."""     window_size = int(0.02 * sr)
    rms = np.sqrt(np.convolve(audio ** 2, np.ones(window_size)/window_size,
mode='same'))
    threshold = threshold_factor * np.median(rms)
audio = audio.copy()     audio[rms < threshold] = 0     return
audio
```

```
 def band_pass_filter(audio, sr, lowcut=200, highcut=3000):
"""Apply band-pass filter for speech frequencies."""     nyquist
= sr / 2     low = lowcut / nyquist     high = highcut / nyquist
    b, a = signal.butter(4, [low, high], btype='band')  return
    signal.filtfilt(b, a, audio)
 def spectral_subtraction(audio, noise_spectrum, sr, frame_size=1024,
hop_size=256, alpha=2.2, beta=0.01):
    """Apply spectral subtraction with noise gate and band-pass filter."""
window = np.hanning(frame_size)
    n_frames = (len(audio) - frame_size) // hop_size + 1
output = np.zeros_like(audio)     overlap = np.zeros_like(audio)
dynamic_beta = beta * np.max(noise_spectrum)     for i in
range(n_frames):         start = i * hop_size
        frame = audio[start:start + frame_size]
if len(frame) < frame_size:              frame = np.pad(frame,
(0, frame_size - len(frame)),
mode='constant')
        frame_windowed = frame * window
spectrum = fft(frame_windowed)
        mag_spectrum = np.abs(spectrum[:frame_size // 2 + 1])
phase_spectrum = np.angle(spectrum[:frame_size // 2 + 1])         clean_mag =
np.maximum(mag_spectrum - alpha * noise_spectrum, dynamic_beta)
clean_spectrum = np.zeros(frame_size, dtype=np.complex128)
clean_spectrum[:frame_size // 2 + 1] = clean_mag * np.exp(1j * phase_spectrum)
        clean_spectrum[frame_size // 2 + 1:] =
np.conj(clean_spectrum[frame_size // 2 - 1:0:-1])         clean_frame
= np.real(ifft(clean_spectrum)) * window         output[start:start +
frame_size] += clean_frame         overlap[start:start + frame_size]
+= window     output /= np.maximum(overlap, 1e-10)
    output = apply_noise_gate(output, sr, threshold_factor=0.25)
output = band_pass_filter(output, sr, lowcut=200, highcut=3000)
max_original = np.max(np.abs(audio))     max_output =
np.max(np.abs(output))     if max_output > 0:          output *=
max_original / max_output     return output
 def compute_snr(clean_audio, noisy_audio, enhanced_audio):
"""Compute SNR using clean audio as reference."""     noise_before
= noisy_audio - clean_audio     noise_after = enhanced_audio -
clean_audio     signal_power = np.mean(clean_audio ** 2)
noise_power_before = np.mean(noise_before ** 2)
noise_power_after = np.mean(noise_after ** 2)
    snr_before = 10 * np.log10(signal_power / noise_power_before) if
noise_power_before > 1e-10 else 0
    snr_after = 10 * np.log10(signal_power / noise_power_after) if
noise_power_after > 1e-10 else 0     return snr_before, snr_after
 def plot_waveform(noisy_audio, enhanced_audio, sr,
filename='waveform_comparison.png'):     time =
np.arange(len(noisy_audio)) / sr
plt.figure(figsize=(12, 6))     plt.subplot(2,
1, 1)  plt.plot(time, noisy_audio, label='Noisy
    Audio')     plt.title('Noisy Audio Waveform')
```

```
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')     plt.grid(True)
plt.legend()     plt.subplot(2, 1, 2)
    plt.plot(time, enhanced_audio, label='Enhanced Audio', color='orange')
plt.title('Enhanced Audio Waveform')     plt.xlabel('Time (s)')
plt.ylabel('Amplitude')     plt.grid(True)     plt.legend()
plt.tight_layout()     plt.savefig(filename)     plt.close()
 def plot_spectrogram(noisy_audio, enhanced_audio, sr,
filename='spectrogram_comparison.png'):
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
    spec, freqs, t, _ = plt.specgram(noisy_audio, Fs=sr, cmap='viridis',
NFFT=1024)     plt.title('Noisy Audio Spectrogram')
plt.xlabel('Time (s)')     plt.ylabel('Frequency
(Hz)')
plt.colorbar(label='dB')     plt.subplot(1,
2, 2)     spec, freqs, t, _ = plt.specgram(enhanced_audio, Fs=sr,
cmap='viridis', NFFT=1024)     plt.title('Enhanced Audio Spectrogram')
plt.xlabel('Time (s)')     plt.ylabel('Frequency
(Hz)')     plt.colorbar(label='dB')
plt.tight_layout()     plt.savefig(filename)     plt.close()
 def plot_snr_comparison(original_snr, enhanced_snr,
filename='snr_comparison.png'):     plt.figure(figsize=(8,
6))
    plt.bar(['Original', 'Enhanced'], [original_snr, enhanced_snr],
color=['blue', 'orange'])     plt.title('SNR Comparison')     plt.ylabel('SNR
(dB)')     plt.grid(True, axis='y')     plt.savefig(filename)     plt.close()
 def process_audio(noisy_file, clean_file, output_file, sr=16000):
    """Process noisy audio, validate with clean audio, and generate plots."""
noisy_audio, sr = load_audio(noisy_file, sr)     clean_audio, _ =
load_audio(clean_file, sr)
    noise_spectrum = estimate_noise_spectrum(noisy_audio, sr,
noise_duration=1.0)
    enhanced_audio = spectral_subtraction(noisy_audio, noise_spectrum, sr)
original_snr, enhanced_snr = compute_snr(clean_audio, noisy_audio,
enhanced_audio)
    sf.write(output_file, enhanced_audio, sr)
plot_waveform(noisy_audio, enhanced_audio, sr)
plot_spectrogram(noisy_audio, enhanced_audio, sr)
plot_snr_comparison(original_snr, enhanced_snr)     return original_snr,
enhanced_snr
 if __name__ == "__main__":
    noisy_file = "noisy_speech.wav"
clean_file = "clean_speech.wav"     output_file
= "enhanced_speech.wav"     original_snr,
enhanced_snr = process_audio(noisy_file,
clean_file, output_file)     print(f"Original
SNR: {original_snr:.2f} dB, Enhanced SNR:
{enhanced_snr:.2f} dB")
```